

MIC Question Bank Solution

Unit 1:

2 Marks Questions

1. State the functions of the following pins: (i) ALE (ii) DT/ $\bar{R}$ (iii) M/ $\bar{IO}$ ' (iv) MN/ $\bar{MX}$ ' (v) Test (vi) Ready (vii) $\bar{Test}$ (viii) A0 (ix) BHE

Ans: (i) ALE: ALE stands for Address Latch Enable. It indicates the availability of valid address on address/ data line

(ii) DT/ $\bar{R}$: Data transmit/receive signal decides the direction of flow of transreceiver. When this pin is high the data is transmitted, when it is low the data is received

(iii) M/ $\bar{IO}$ ': It is used in minimum mode, to distinguish between a memory operation and I/O operation. When this pin is low it indicates that the Read/Write operation is done using I/O devices, when it is high it indicates that the Read/Write operation is done using Memory

(iv) MN/ $\bar{MX}$ ': This is an input pin to the microprocessor which indicates the processor will work in which mode. If it is high the processor works in minimum mode, if it is low the processor will work in maximum mode

(v) Test: Test pin is an active low input line dedicated for 8087 math co-processor. When the processor is busy executing instructions it sends high signal on the TEST pin, low signal indicates that the processor is in idle state.

(vi) Ready: It is active high and indicates that the device is ready to transfer data, when it is low it indicates the wait state. The main use of this pin is to insert wait states into the timing of the microprocessor

(vii) $\bar{Test}$: Test pin is an active low input line dedicated for 8087 math co-processor. When the processor is busy executing instructions it sends high signal on the TEST pin, low signal indicates that the processor is in idle state.

(viii) A0: It is the lowest bit of address/data line. In the first clock cycle it carries the first bit of the address from the processor.

(ix) BHE: BHE is active low signal and it stands for Bus High Enable. It indicates the transfer of data using data bus D8- D15. This signal is low during the first clock cycle.

2. What is queue in 8086

Ans: The 8086 has 6-byte prefetch instruction queue which is used to prefetch upto 6-bytes of instruction from memory and store those instructions before execution. It helps in pipelining, which makes the execution of processor faster

3. State any four features of 8086

- Ans:**
- i) It is a 16-bit microprocessor
 - ii) It is a single core processor
 - iii) It has two separate units: BIU and EU
 - iv) It has 20-bit address bus and can access up to 1MB of memory

4 Marks Questions

1. Describe Memory segmentation in 8086 and list its advantages.

Ans: (Diagram 2M, Description 1M, Any 2 Advantage 1M)

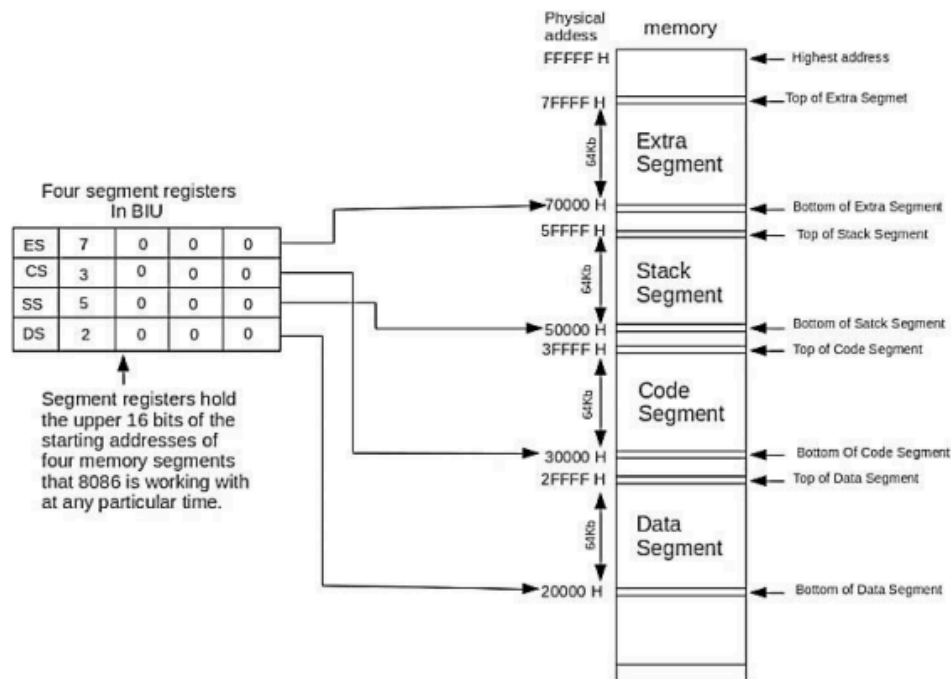


Fig: Memory Segmentation in 8086

- Segmentation is the process in which the main memory of the computer is divided into different segments and each segment has its own base address
- It is basically used to enhance the speed of execution of computer system
- 8086 has 1MB of memory which is divided into four segments
- The four segments are of 64kB each and are made of contiguous memory locations
- The four segments are Code Segment, Data Segment, Stack Segment, Extra Segment
- The **Code Segment** is used to store the executable instructions in the memory
- The **Data Segment** is used to store the data required by the processor at the time of execution

- The **Stack Segment** is used to store the stack type of data structure
- The **Extra Segment** is used to store the destination string at the time of performing string instruction

Advantage: (any two)

- It provides a powerful memory management mechanism
- It allows processor to easily share data
- Since the data fetching is easier the execution time of processor is optimized
- Data related or Stack Related operations can be performed in different segments

2. Explain the concept of pipelining in 8086. State the advantages and disadvantages of pipelining

Ans: (Explanation 2M, Diagram 1M, Advantages 1M)

- The process of fetching the next instruction when the present instruction is being executed is called as pipelining
- Pipelining has become possible due to the use of 6 byte prefetch instruction queue
- Instructions are stored in memory, therefore, it has to be fetched, decoded and then executed
- Pipelining boosts performance as fetching and executing are done simultaneously
- BUI fills in the queue until the entire queue is full
- BIU restarts filling in the queue when atleast two location of queue are vacant

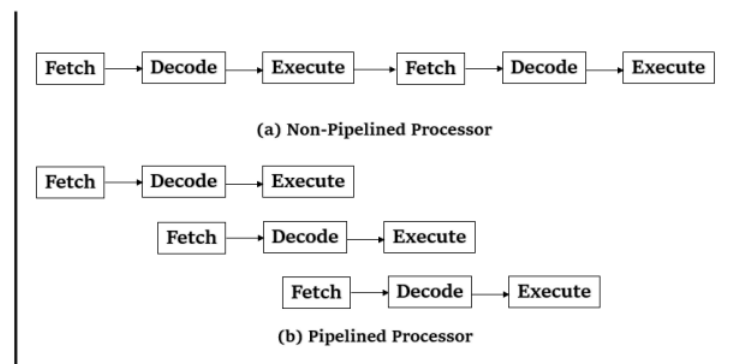


Fig: Concept of Pipelining

Advantage of Pipeline:

- It boosts the performance of the processor
- Fetching and executing are done alongside at the same time
- Fewer instruction cycles are required to execute the instruction
- It saves overall time of the processor

3. Draw and explain the flag register of 8086.

Ans: (Correct Diagram 1M, Status Flags 2M, Control Flags 1M)

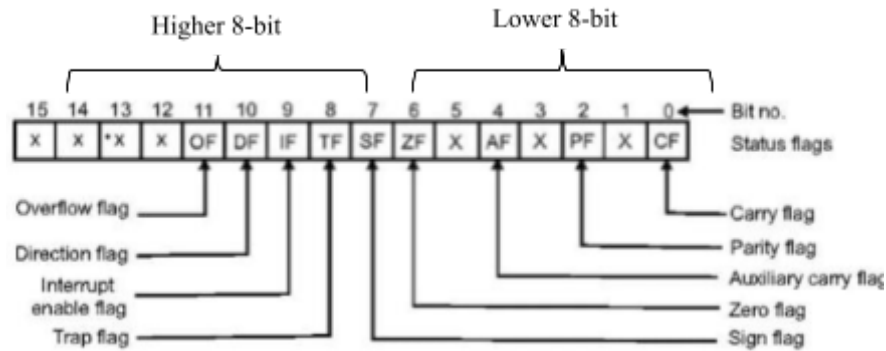


Fig: Flag Register of 8086

- The 8086 flag register contents indicate the results of computations in the ALU
- It also contains flag bits to control the CPU operation

Status Flag:

- 1. Overflow Flag (OF):** This flag is set if an overflow occurs i.e. if the result of a signed operation is large enough to accommodate in a destination register
- 2. Sign Flag (SF):** This flag is set if the result of arithmetic operation is a signed number.
- 3. Zero Flag (ZF):** After any arithmetic or logical operation if the result is 0, the zero flag is set to 1
- 4. Auxiliary Carry Flag (AC):** This flag is generally used in BCD number system, whenever the lower bit addition generates a carry and pass on to the upper bit this flag gets set to 1. This is the only flag register that is not accessible by the programmer
- 5. Parity Flag (PF):** If after any arithmetic or logical operation the result has even parity (Even number of '1s') this flag is set to 1
- 6. Carry Flag (CF):** When a carry is generated after arithmetic operation this flag is set to 1.

Control Flags:

- 1. Direction Flag (DF):** This flag is used in string instruction. If the direction flag is set to 1, then string is accessed from higher memory location towards lower memory location. If direction flag is reset (0), then the string is accessed from lower memory location to higher memory location
- 2. Interrupt Flag (IF):** If interrupt flag is set to 1, the microprocessor will recognize interrupt request. If Interrupt flag is reset (0), the microprocessor will not recognize any interrupt request and will ignore them.

3. Trap Flag (TF): When this flag is set to 1, the microprocessor executes instruction in single step mode for debugging. In single step mode, the CPU automatically generates an internal interrupt after each instruction, allowing program to be inspected instruction by instruction.

6 Marks Questions

1. Define logical and effective address. Describe Physical Address generation in 8086.

If CS = 2135H and IP = 3478H, calculate Physical Address

Ans: (Each Define 1M, Physical Address Generation 2M, Numerical 2M)

Logical Address: Logical address are also known as virtual address. It is generated by the CPU during program execution. Logical addresses provide a way for the CPU to access different locations in memory without needing to know the physical organization of the memory.

Effective Address: The offset for a memory operand is called the operand's effective address or EA. It is an unassigned 16 bit number that expresses the operand's distance in bytes from the beginning of the segment in which it resides.

Generation of 20 bit physical address in 8086:-

1. Segment registers carry 16 bit data, which is also known as base address.
2. BIU appends four 0 bits to LSB of the base address. This address becomes 20- bit Address.
3. Any base/pointer or index register carries 16 bit offset.
4. Offset address is added into 20-bit base address which finally forms 20 bit physical address of memory location

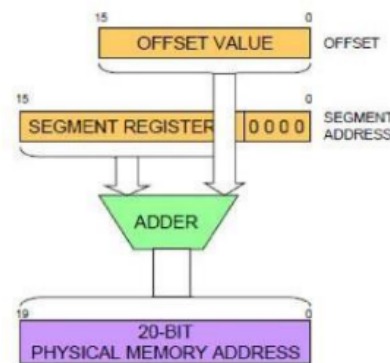


Fig: Physical Address Generation

Given: SA = 2135H

OffAdd = 3478H

Formula: PA = SA * 10H + OffAdd

= 2135H * 10H + 3478H

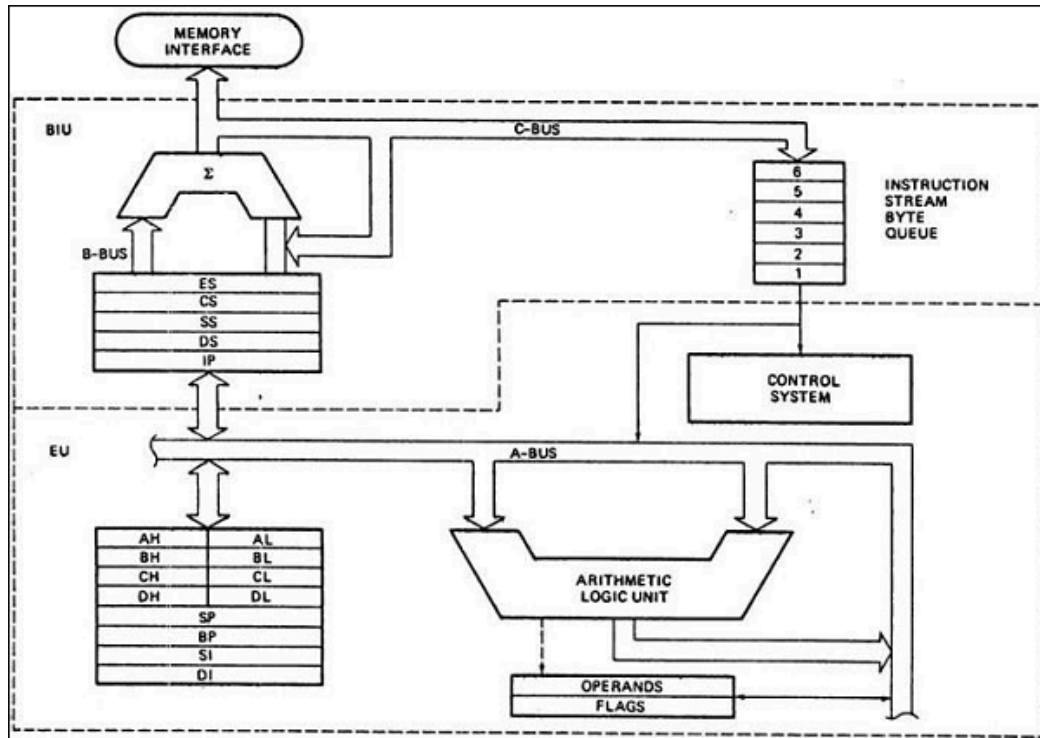
$$= 21350 + 3478$$

$$= 247C8H$$

$$\therefore PA = 247C8H$$

2. Draw architectural block diagram of 8086 and describe its register organization.

Ans:



Unit 2:

2 Marks Questions

1. Write the algorithm for any program given
2. Draw the flowchart for any program given (Any algorithm/ flowchart from addition, subtraction, multiplication or division)

4 Marks Questions

1. Describe how an assembly language program is developed and debugged using system tools such as editors, assemblers, linkers and debuggers.

Ans: (Each tool explanation 1M)

Assembly language development tools:

Editor:

It is a program which helps to construct assembly language programs with a file extension .asm, in right format so that the assembler will translate it to machine language. It enables one to create, edit, save, copy and make modification in the source file.

Assembler:

Assembler is a program that translates assembly language programs to the correct binary code. It also generates the file called as object file with extension .obj. It also displays syntax errors in the program, if any.

Linker:

It is a programming tool used to convert Object code (.OBJ) into an executable (.EXE) program. It combines, if requested, more than one separated assembled module into one executable module such as two or more assembly programs or an assembly language with C program.

Debugger:

Debugger is a program that allows the execution of a program in single step mode under the control of the user. The errors in the program can be located and corrected using a debugger. Debugger generates .exe file.

2. Describe the directives used to define the procedures with suitable example

Ans: (2M for each directive explanation)

The directives used in processor are PROC and ENDP

PROC Directive: It indicates the start of the procedure. It is used to assign the label to the procedure. The procedure are of two types NEAR procedure and FAR procedure. The proc directive is usually followed by the keyword NEAR or FAR based on the type of procedure that is being used in the Program. NEAR Procedure are the procedure which are declared in the same code segment from which it is called and FAR procedure are the procedure which are declared in the different code segment then from the ones it is called

Example: Proc_name PROC Near

;it indicates the start of the procedure

ENDP Directive: It indicates the end of the procedure. The ENDP (End Procedure) assembler directive marks the conclusion of a subroutine or procedure that was initiated with the PROC directive in assembly language programming. It is essential for defining the scope of a procedure, allowing the assembler to manage return addresses and procedure-specific instructions correctly.

Example: ENDP Proc_name ; It indicates the end of the procedure

6 Marks Questions

1. Demonstrate in detail the program development steps in assembly language programming.

Ans: (Each Correct Step 1M)

(i) Defining the Problem: The first step in writing a program is to think carefully about the problem that you want the program to solve.

(ii) Writing Algorithm: The sequence which your program should follow can be specified as a series of steps or in other words sequence of operation.

(iii) Flowchart: The flowchart is a graphical representation of the program operation or task. Each specific operation is specified by different symbols. The lists of symbols are given below:

Symbol	Name	Function
	Start/end	An oval represents a start or end point.
	Arrows	A line is a connector that shows relationships between the representative shapes.
	Input/Output	A parallelogram represents input or output.
	Process	A rectangle represents a process.
	Decision	A diamond indicates a decision.

(iv) Initialization Checklist: There may be many variables and segment registers which need to be defined in the program, but before defining they need to be initialized. The best way to approach the initialization task is to make a checklist of entire variables and registers which are going to be used in the program.

(v) Choosing Instructions: Proper instructions which are required in the program are chosen. Which means that the entire instruction set must be known by the programmer.

(vi) Converting Algorithms to Assembly Language Programs: Once you've selected the instructions for the operations to be performed then arrange these instructions in sequence as per algorithm so that desired output must be obtained after execution. First, we declare the data in the data segment, and then we write the actual program in the code segment.

2. Explain the following assembler directives

(Any six of the following will be asked)

Ans: (Each correct explanation with proper example 1M)

(i) DB

The DB directive is used to reserve a byte or bytes of memory location in the available memory.

Eg: num1 DB 20H ; assembler reserves 1 byte of memory for the variable named num1 and initializes its value to 20

(ii) DW

The DW directive is used to reserve a word or words of memory location in the available memory.

Eg: num1 DW 1234H ; this makes the assembler reserve a word (16-bit) in memory

(iii) DD

This directive is used to direct the assembler to reserve four words (8 bytes) of memory for the specified variable.

Eg: num DD 1234567812345678H ; this makes the assembler reserve 4 words (16x4=64-bit) in the memory

(iv) EQU

This directive tells the assembler to assign a value to the corresponding label. It is used to assign a constant value to a label in a program. When the label is called from the program it is replaced with the value assigned to it by EQU directive. It is used when a certain value needs to be used many times in program

Eg: Count EQU 1000H ; it assigned the value 1000 to the label Count

(v) DUP

- It can be used to initialize several locations in a memory segment of the same type with same values

- It is used to create duplicates in the memory

Eg: b1 DB 5 DUP (?)

This creates 5 blank values in the memory of 8-bit each with the

(vi) SEGMENT

The segment directive is used to indicate the start of a logical segment.

Eg: CODE SEGMENT statement indicates to the assembler the start of a logical segment called CODE

(vii) ASSUME

The ASSUME directive is used to inform the assembler the names of the logical segments to be assumed for different segments used in the program. It is used to pass the logical address in the program

Eg: The statement ASSUME CS:CODE directs the assembler that the machine code (executable program) are available in a segment named CODE and hence CS register is to be loaded with the segment address for the label CODE, while loading.

(viii) GLOBAL

- This directive is used to declare a variable that can be used by multiple code segments in a program

- The labels, variables, constants or procedures declared GLOBAL may be used by other code segment

Eg: ROUTINE PROC GLOBAL

This instruction declares a procedure named routine which can be accessed by any code segment in the program

Unit 3:

1. Machine Language Instruction format in 8086

Ans:

Machine Language Instruction format is as follows:

1. One Byte Instruction
2. Register to Register
3. Register to/from memory with no displacement
4. Register to/from memory with displacement
5. Immediate Operand to Register
6. Immediate Operand to Memory with 16-bit displacement

1. One Byte Instruction:

- This format is only one byte long and may have the implied data or register operands.
- The least significant 3-bits of the opcode are used for specifying the register operand, if any.
- Otherwise, all the 8 bits form an opcode and the operands are implied
- Example: DAA, CLD, INT, MUL BL



Fig: One byte instruction format

2. Register to Register:

- This format is 2 bytes long
- The first byte of the code specifies the operation code and width of the operand specified by the 'W' bit
- The second byte of the code shows the register operands and R/M field, as show below
- The register represented by the REG field is one of the operands
- The R/M field specifies another register or memory location i.e. the other operand
- Example: ADD AL,BL, MOV BL,AL

D7	D1	D0	D7 D6	D5 D4 D3	D2 D1 D0
OPCODE			11	REG	R/M

Fig: Register to Register Instruction Format

3. Register to/from memory with no displacement:

- This format is also 2 bytes long and similar to the Register to Register format except for the MOD field
- The MOD field shows the mode of addressing used
- Example: MOV AX,ans

D7	D1	D0	D7 D6	D5 D4 D3	D2 D1 D0
OPCODE			MOD	REG	R/M

Fig: Register to/from memory with no displacement

4. Register to/from Memory with Displacement:

- This type of instruction format contains 1 or 2 additional bytes for displacement along with 2 byte format of the register to/from memory without displacement

- The format is shown below:

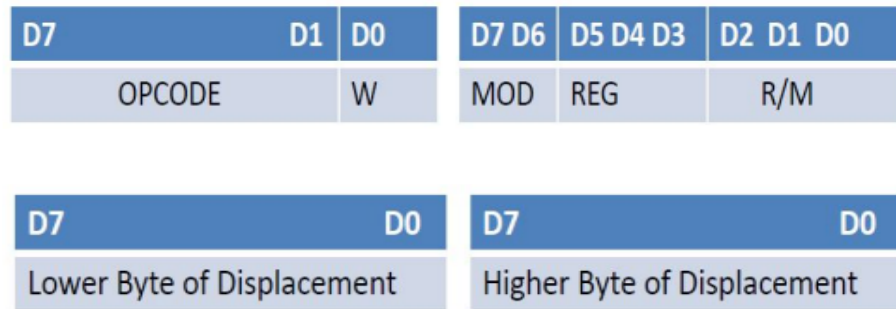


Fig: Register to/from Memory with Displacement

5. Immediate Operand to Register:

- In this format, the first byte as well as the 3-bits from the second byte which are used for REG field in case of register to register format are used for opcode
- It also contains one or two bytes of immediate data
- The complete instruction format is shown below:

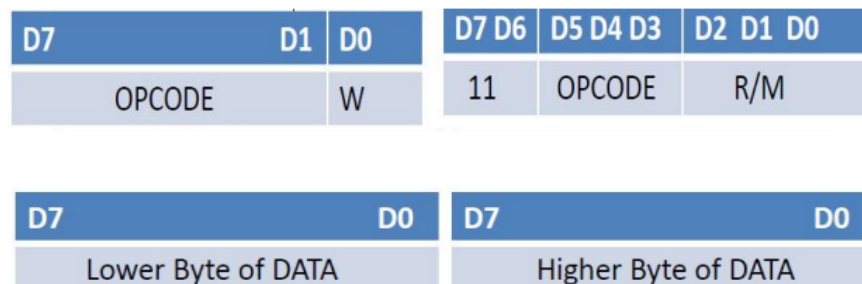


Fig: Immediate Operand to Register Instruction format

6. Immediate Operand to Memory with 16-bit displacement:

- This type of instruction format requires 5 or 6 bytes for coding
- The first 2 bytes contain the information regarding OPCODE, MOD and R/M fields
- The remaining 4 bytes contain 2 bytes of displacement and 2 bytes of data

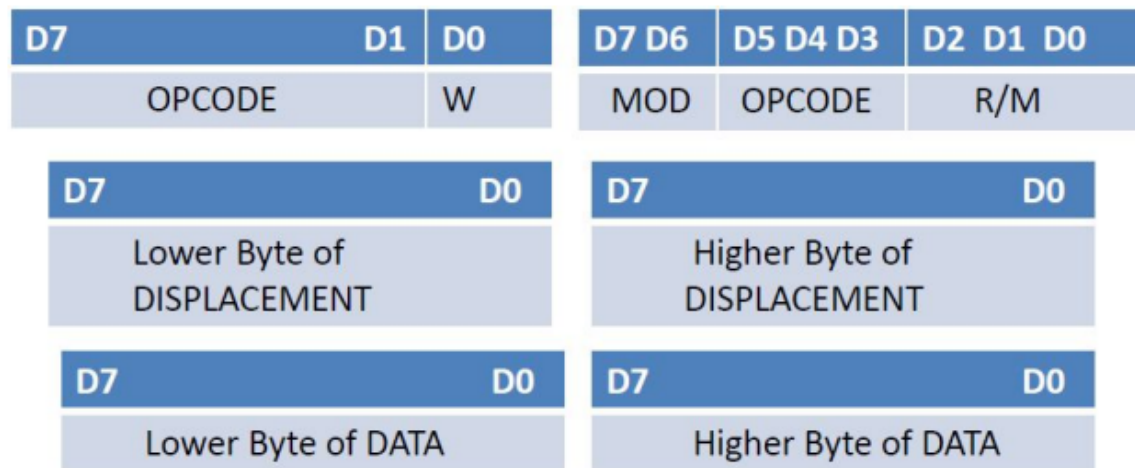


Fig: Immediate Operand to Memory with 16-bit displacement

2. Identify the addressing modes of given instructions

(Refer to the notes given in class)